
Lab Assignment 17.1

HEPSIBA DEVARA
2403A51L25
BATCH-51

Task 1:

Prompt: Generate a Python Program for cleaning a customer dataset. That Handles missing values in columns (age, income, city), remove duplicate records, standardize text values (e.g., city names in lowercase) and encode categorical variables (gender, city).

Code:

```
import pandas as pd
dataset=pd.read_csv('/content/shopping_trends.csv')
display(dataset.head())
df = dataset.copy() # Work on a copy to preserve the original 'dataset'
# Check and fill missing 'Age' with the mean
if 'Age' in df.columns and df['Age'].isnull().any():
    df['Age'].fillna(df['Age'].mean(), inplace=True)
    print("Filled missing 'Age' values with mean.")
# Check and fill missing 'Purchase Amount (USD)' (proxy for income) with the mean
if 'Purchase Amount (USD)' in df.columns and df['Purchase Amount (USD)'].isnull().any():
    df['Purchase Amount (USD)'].fillna(df['Purchase Amount (USD)'].mean(), inplace=True)
    print("Filled missing 'Purchase Amount (USD)' values with mean.")
# Check and fill missing 'Location' (proxy for city) with the mode
if 'Location' in df.columns and df['Location'].isnull().any():
    # Ensure column is string type before finding mode and filling
    df['Location'] = df['Location'].astype(str)
    mode_location = df['Location'].mode()[0]
    df['Location'].fillna(mode_location, inplace=True)
    print("Filled missing 'Location' values with mode.")
print("\nDataFrame after handling missing values (if any):")
initial_rows = df.shape[0]
df.drop_duplicates(inplace=True)
final_rows = df.shape[0]
print(f"Removed {initial_rows - final_rows} duplicate rows.")
print("\nDataFrame after removing duplicates:")
if 'Location' in df.columns:
    df['Location'] = df['Location'].astype(str).str.lower()
    print("Standardized 'Location' names to lowercase.")
```

```

print('Standardized Location names to lowercase.')
else:
    print("'Location' column not found for standardization.")
print("\nDataFrame after standardizing text values (if 'Location' exists):")
cols_to_encode = []
if 'Gender' in df.columns: cols_to_encode.append('Gender')
if 'Location' in df.columns: cols_to_encode.append('Location')
if cols_to_encode:
    df_encoded = pd.get_dummies(df, columns=cols_to_encode, drop_first=True)
    print(f"Encoded categorical variables: {'', '.join(cols_to_encode)}".")
else:
    df_encoded = df.copy()
    print("No specified categorical columns ('Gender', 'Location') found for encoding.")
print("\nDataFrame after encoding categorical variables:")
display(df.head())

```

Output:

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	Review Rating	Subscription Status	Payment Method	Shipping Type	Discount Applied	Promo Code Used	Previous Purchases	Preferred Payment Method	Frequency of Purchases
0	1	25	Male	Skate	Clothing	53	Kentucky	L	Gray	Winter	3.1	Yes	Credit Card	Express	Yes	Yes	14	Visa	Frequently
1	2	19	Male	Sneaker	Clothing	64	Maine	L	Maroon	Winter	3.1	Yes	Bank Transfer	Express	Yes	Yes	2	Cash	Frequently
2	3	30	Male	Jean	Clothing	73	Massachusetts	S	Maroon	Spring	3.1	Yes	Cash	Free Shipping	Yes	Yes	23	Credit Card	Weekly
3	4	21	Male	Sandals	Footwear	90	Rhode Island	M	Maroon	Spring	3.5	Yes	PayPal	Next Day Air	Yes	Yes	49	PayPal	Weekly
4	5	45	Male	Skate	Clothing	49	Oregon	M	Turquoise	Spring	2.7	Yes	Cash	Free Shipping	Yes	Yes	31	PayPal	Annually

DataFrame after handling missing values (if any):
 Removed 0 duplicate rows.
 DataFrame after removing duplicates:
 Standardized 'location' names to lowercase.
 DataFrame after standardizing text values (if 'location' exists):
 Encoded categorical variables: gender, location.
 DataFrame after encoding categorical variables:

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	Review Rating	Subscription Status	Payment Method	Shipping Type	Discount Applied	Promo Code Used	Previous Purchases	Preferred Payment Method	Frequency of Purchases
0	1	25	Male	Skate	Clothing	53	kentucky	L	Gray	Winter	3.1	Yes	Credit Card	Express	Yes	Yes	14	Visa	Frequently
1	2	19	Male	Sneaker	Clothing	64	maine	L	Maroon	Winter	3.1	Yes	Bank Transfer	Express	Yes	Yes	2	Cash	Frequently
2	3	30	Male	Jean	Clothing	73	massachusetts	S	Maroon	Spring	3.1	Yes	Cash	Free Shipping	Yes	Yes	23	Credit Card	Weekly
3	4	21	Male	Sandals	Footwear	90	rhode island	M	Maroon	Spring	3.5	Yes	PayPal	Next Day Air	Yes	Yes	49	PayPal	Weekly
4	5	45	Male	Skate	Clothing	49	oregon	M	Turquoise	Spring	2.7	Yes	Cash	Free Shipping	Yes	Yes	31	PayPal	Annually

Explanation:

This program loads your shopping_trends.csv dataset, then cleans it by filling missing 'Age' and 'Purchase Amount (USD)' with their means, and 'Location' with its mode. It removes duplicate rows, standardizes 'Location' names to lowercase, and finally encodes 'Gender' and 'Location' using one-hot encoding for further analysis.

Task 2:

Prompt: Preprocess sales transaction data that Convert date columns into datetime format, create new features (month, year, day of week), normalize sales amount column and detect and handle outliers in dataset.

Code:

```

import pandas as pd
from sklearn.preprocessing import MinMaxScaler
data_set=pd.read_csv('/content/sales_data.csv')
display(data_set.head())
df_sales = data_set.copy()
if 'Sale_Date' in df_sales.columns:
    df_sales['Sale_Date'] = pd.to_datetime(df_sales['Sale_Date'])
    df_sales['Month'] = df_sales['Sale_Date'].dt.month
    df_sales['Year'] = df_sales['Sale_Date'].dt.year
    df_sales['DayOfWeek'] = df_sales['Sale_Date'].dt.dayofweek # Monday=0, Sunday=6
    print("Converted 'Sale_Date' column to datetime and created 'Month', 'Year', 'DayOfWeek' features:")
    display(df_sales[['Sale_Date', 'Month', 'Year', 'DayOfWeek']].head())
else:
    print("No 'Sale_Date' column found in `sales_data.csv` to convert or extract features from.")
# Assuming 'Sales_Amount' as the column to normalize
sales_column = 'Sales_Amount' # Adjust if your sales amount column has a different name
if sales_column in df_sales.columns:
    scaler = MinMaxScaler()
    df_sales[f'{sales_column}_normalized'] = scaler.fit_transform(df_sales[[sales_column]])
    print(f"Normalized '{sales_column}' column (to '{sales_column}_normalized'):")
    display(df_sales[[sales_column, f'{sales_column}_normalized']].head())
else:
    print(f"No '{sales_column}' column found for normalization.")
sales_column = 'Sales_Amount' # Adjust if your sales amount column has a different name
if sales_column in df_sales.columns:
    Q1 = df_sales[sales_column].quantile(0.25)
    Q3 = df_sales[sales_column].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    outliers_count = ((df_sales[sales_column] < lower_bound) | (df_sales[sales_column] > upper_bound)).sum()
    print(f"Number of outliers detected in '{sales_column}': {outliers_count}")
    # Cap outliers
    df_sales[f'{sales_column}_capped'] = df_sales[sales_column].clip(lower=lower_bound, upper=upper_bound)
    print(f"Outliers in '{sales_column}' have been capped. Original vs. Capped values:")
    display(df_sales[[sales_column, f'{sales_column}_capped']].head())
else:
    print(f"No '{sales_column}' column found for outlier detection.")
print("\nFinal preprocessed `sales_data.csv` DataFrame head:")
display(df_sales.head())

```

Output:

Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	Product_Category	Unit_Cost	Unit_Price	Customer_Type	Discount	Payment_Method	Sales_Channel	Region_and_Sales_Rep	
0	1002	2023-03-01	Bob	North	5053.97	18	Furniture	162.75	267.22	Returning	0.09	Cash	Online	North-Bob
1	1093	2023-04-21	Bob	West	4384.02	17	Furniture	3816.39	4209.44	Returning	0.11	Cash	Retail	West-Bob
2	1015	2023-09-21	David	South	4631.23	30	Food	261.56	371.40	Returning	0.20	Bank Transfer	Retail	South-David
3	1072	2023-08-24	Bob	South	2167.94	39	Clothing	4330.03	4467.75	New	0.02	Credit Card	Retail	South-Bob
4	1061	2023-03-24	Charlie	East	3750.20	13	Electronics	637.37	692.71	New	0.08	Credit Card	Online	East-Charlie

Converted 'Sale_Date' column to datetime and created 'Month', 'Year', 'DayOfWeek' features:

	Sale_Date	Month	Year	DayOfWeek
0	2023-03-01	3	2023	4
1	2023-04-21	4	2023	4
2	2023-09-21	9	2023	3
3	2023-08-24	8	2023	3
4	2023-03-24	3	2023	4

Normalized 'Sales_Amount' column (to 'Sales_Amount_normalized'):

	Sales_Amount	Sales_Amount_normalized
0	5053.97	0.500950
1	4384.02	0.433202
2	4631.23	0.458204
3	2167.94	0.209105
4	3750.20	0.369108

Number of outliers detected in 'Sales_Amount': 0
Outliers in 'Sales_Amount' have been capped. Original vs. Capped values:

	Sales_Amount	Sales_Amount_capped
0	5053.97	5053.97
1	4384.02	4384.02
2	4631.23	4631.23
3	2167.94	2167.94
4	3790.20	3790.20

Final preprocessed 'sales_data.csv' DataFrame head:

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	Product_Category	Unit_Cost	Unit_Price	Customer_Type	Discount	Payment_Method	Sales_Channel	Region_and_Sales_Rep	Month	Year	DayOfWeek	Sales_Amount
0	1182	2023-02-08	Bob	North	5053.97	18	Furniture	152.75	280.22	Returning	0.03	Cash	Online	North-Bob	2	2023	4	5053.97
1	1093	2023-04-21	Bob	West	4384.02	17	Furniture	3816.39	4209.44	Returning	0.11	Cash	Retail	West-Bob	4	2023	4	4384.02
2	1015	2023-05-21	David	South	4631.23	38	Food	351.56	371.40	Returning	0.20	Bank Transfer	Retail	South-David	5	2023	2	4631.23
3	1072	2023-08-24	Bob	South	2167.94	20	Clothing	4330.03	4457.75	New	0.02	Credit Card	Retail	South-Bob	8	2023	3	2167.94
4	1061	2023-10-24	Charlie	East	3790.20	13	Electronics	637.37	692.71	New	0.08	Credit Card	Online	East-Charlie	3	2023	4	3790.20

Explanation:

The code preprocessed sales_data.csv. It converted the Sale_Date column to datetime, extracting Month, Year, and DayOfWeek. It then normalized the Sales_Amount column using MinMaxScaler.

Task 3:

Prompt: Generate SQL queries for basic CRUD operations:

- Retrieve all courses
- Update a student’s email
- Delete an enrollment record
- Validate the changes using SELECT statements.

Code:

```
-- Retrieve all courses
SELECT * FROM Courses;

-- Update a student's email
UPDATE Students SET email = 'new.email@example.com' WHERE student_id = 1;

-- Delete an enrollment record
DELETE FROM Enrollments WHERE enrollment_id = 1;

-- Validate the changes using SELECT statements
SELECT * FROM Students WHERE student_id = 1;
SELECT * FROM Enrollments WHERE enrollment_id = 1;
```

Output:

```
sqlite> SELECT * FROM Courses;
1|Mathematics|MATH101
2|Physics|PHYS101
3|Chemistry|CHEM101
4|Biology|BIOL101
sqlite> UPDATE Students SET email = 'new.email@example.com' WHERE student_id = 1;
sqlite> DELETE FROM Enrollments WHERE enrollment_id = 1;
sqlite> SELECT * FROM Students WHERE student_id = 1;
1|John|Doe|new.email@example.com
sqlite> SELECT * FROM Enrollments WHERE enrollment_id = 1;
```

Explanation:

CRUD operations refer to the four basic functions of persistent storage: Create, Read, Update, and Delete. In the context of the provided SQL script:

- Create: This is demonstrated by the INSERT statements, which add new records to the Students, Courses, and Enrollments tables.

- Read: This is shown by the SELECT statements, which retrieve data from the tables. For example, the query to retrieve all courses and the queries to validate changes after updating a student's email and deleting an enrollment record.
- Update: This is illustrated by the UPDATE statement, which modifies the email of a student in the Students table.
- Delete: This is represented by the DELETE statement, which removes a specific enrollment record from the Enrollments table.

Task 4:

Prompt: Write aggregate queries to count number of students enrolled per course and display courses with more than one student enrolled. Analyse the correctness of results.

Code:

```
-- Count number of students enrolled per course
SELECT course_id, COUNT(student_id) AS student_count
FROM Enrollments
GROUP BY course_id;
-- Display courses with more than one student enrolled
SELECT course_id, COUNT(student_id) AS student_count
FROM Enrollments
GROUP BY course_id
HAVING COUNT(student_id) > 1;
```

Output:

```
sqlite> SELECT course_id, COUNT(student_id) AS student_count FROM Enrollments
GROUP BY course_id;
1|1
2|1
3|1
4|1
sqlite> SELECT course_id, COUNT(student_id) AS student_count FROM Enrollments
GROUP BY course_id HAVING COUNT(student_id) > 1;
```

Explanation:

The first query will return the number of students enrolled in each course. The second query will return only those courses that have more than one student enrolled. The correctness of the results can be verified by cross-referencing the enrollment data in the Enrollments table with the course information in the Courses table. If the counts match the expected values based on the inserted data, then the queries are correct. Additionally, checking for any discrepancies or unexpected results can help identify any issues with the data or the queries themselves.

Task 5:

Prompt: Rewrite inefficient sub queries using JOINS and test both queries for correctness.

Sample input Code:

```
SELECT * FROM Students
WHERE student_id IN (
SELECT student_id FROM Enrollments WHERE course_id = 101
);
```

Code:

```
-- Rewritten Query using JOIN
SELECT s.* FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id
WHERE e.course_id = 101;
```

Output:

```
sqlite> SELECT s.* FROM Students s JOIN Enrollments e ON s.student_id = e.student_id WHERE e.course_id = 101;
sqlite> 
```

Explanation:

The Sample query uses a subquery to find student IDs enrolled in a specific course, which can be inefficient as it may require scanning the Enrollments table multiple times. The rewritten query uses a JOIN to directly connect the Students and Enrollments tables, allowing for a more efficient retrieval of the desired records. Both queries should return the same results, but the JOIN version is typically faster, especially as the size of the tables increases.